Core/Src/main.c

```c
/* CPE 3160
 * John Oliver
 *
 * P1 - Lock
 * Chris Amey and Alan Kusparmakov
 *
 * Video:
 * https://drive.google.com/file/d/1hArrJB_lnd-JMD0XwG729pXTaAC3bJnJ/view?usp=sharing
 *
 * Schematic:
 * https://drive.google.com/file/d/1SX4DZLqecqLpMrKSfqTl6fLLpEubJk_C/view?usp=sharing
 */

#include "init.h"
#include "keypad.h"
#include "led.h"
#include "lcd.h"
#include "stdbool.h"
#include "stm32l476xx.h"
#include "stm32l4xx_hal.h"
#include <stddef.h>
#include <stdint.h>
#include <string.h>

#define INPUT_STAR 10
#define INPUT_POUND 15

#define LINE_WIDTH 16


void welcome_message() {
    LCD_print("Welcome!", FIRST);
    LCD_print("Enter key here", SECOND);
}

void init()
{
  gpio_init();
  LCD_init(true, false, true, true, true, false);
}

enum State { LOCKED, UNLOCKED, CHANGING };

void status_message(enum State state, char *input)
{
  switch (state) {
      case (LOCKED):
        LCD_clear();
            LCD_print("LOCKED", FIRST);
        LCD_print(input, SECOND);
          LED_turn_on();
            break;

        case (UNLOCKED):
        LCD_clear();
            LCD_print("UNLOCKED", FIRST);
            LED_turn_off();
            break;

        case (CHANGING):
            LCD_clear();
            LCD_print("ENTER PASSWORD", FIRST);
            LCD_print(input, SECOND);
            break;
  }
}

void buffer_append(char *buffer, char ch)
{
  int len = strlen(buffer);
  buffer[len] = ch;
}

void buffer_backspace(char *buffer)
{
  int len = strlen(buffer);
  if (len == 0)
    return;

  buffer[len - 1] = 0;
}

void buffer_reset(char *buffer)
{
  int len = strlen(buffer);
  memset(buffer, 0, len);
}

int main()
{
  init();
  welcome_message();
```

```
  LED_turn_on();

  HAL_Delay(3000);

  // enough space for longest string and a null terminator
  char password[LINE_WIDTH + 1] = { 0 };
  char input[LINE_WIDTH + 1] = { 0 };

  strcpy(password, "1234");

  int keyInput = 0;

  enum State state = LOCKED;

  while (1) {
    status_message(state, input);

    keyInput = get_pressed_key();
    while (keyInput == INVALID_VALUE) {
      keyInput = get_pressed_key();
      HAL_Delay(25);
      continue;
    }

    switch (state) {
    case LOCKED:
      if (keyInput == INPUT_STAR) {
        buffer_reset(input);
        continue;
      }

      if (keyInput == INPUT_POUND) {
        buffer_backspace(input);
        continue;
      }

      buffer_append(input, '0' + keyInput);

      if (strcmp(input, password) == 0) {
        state = UNLOCKED;
      }

      break;

    case UNLOCKED:
      // buffer is empty while unlocked
      memset(input, 0, sizeof(input));

      if (keyInput == INPUT_STAR) {
        state = LOCKED;
        continue;
      }

      if (keyInput == INPUT_POUND) {
        state = CHANGING;
        continue;
      }

      break;

    case CHANGING:
      // change password
      int len = strlen(input);
      if (keyInput == INPUT_POUND) {
        if (len == 0) {
          state = LOCKED;
        }

        buffer_backspace(input);
        continue;
      }

      if (keyInput == INPUT_STAR) {
        if (len < 4)
          continue;

        strcpy(password, input);
        buffer_reset(input);
        state = LOCKED;
        continue;
      }

      buffer_append(input, '0' + keyInput);
    }
  }
}
```

Core/CMakeLists.txt

```
target_include_directories(${CMAKE_PROJECT_NAME} PRIVATE Inc)
add_subdirectory(Src)
```

Core/Inc/init.h

```
// lcd.h
#ifndef init_h
#define init_h

void gpio_init();

#endif
```

Core/Inc/keypad.h

```
// keypad.h
#ifndef keypad_h
#define keypad_h

/* On the Keypad, from left to right: Row1, Row2, Row3, Row4, Col.1, Col.2, Col.3
*/

#define ROW_COUNT 4
#define COL_COUNT 3

#define PA_ROW_START 5
#define PC_COLUMN_START 10

#define INVALID_VALUE -1

int get_pressed_key();

#endif
```

Core/Inc/lcd.h

```c
// lcd.h
#ifndef lcd_h
#define lcd_h

#include "stdbool.h"

#define DDRAM_ADDRESS_SECOND_LINE 0x40
#define DDRAM_ADDRESS_FIRST_LINE 0x0

#define INSTRUCTION_DISPLAY_CLEAR_Pos 0
#define INSTRUCTION_CURSOR_HOME_Pos 1
#define INSTRUCTION_ENTRY_MODE_SET_Pos 2
#define INSTRUCTION_DISPLAY_ON_OFF_Pos 3
#define INSTRUCTION_CURSOR_SHIFT_Pos 4
#define INSTRUCTION_FUNCTION_SET_Pos 5

#define INSTRUCTION_DDRAM_ADDRESS_SET_Pos 7

#define INSTRUCTION_DISPLAY_CLEAR_Msk (1 << INSTRUCTION_DISPLAY_CLEAR_Pos)
#define INSTRUCTION_CURSOR_HOME_Msk (1 << INSTRUCTION_CURSOR_HOME_Pos)
#define INSTRUCTION_ENTRY_MODE_SET_Msk (1 << INSTRUCTION_ENTRY_MODE_SET_Pos)
#define INSTRUCTION_DISPLAY_ON_OFF_Msk (1 << INSTRUCTION_DISPLAY_ON_OFF_Pos)
#define INSTRUCTION_CURSOR_SHIFT_Msk (1 << INSTRUCTION_CURSOR_SHIFT_Pos)
#define INSTRUCTION_FUNCTION_SET_Msk (1 << INSTRUCTION_FUNCTION_SET_Pos)
#define INSTRUCTION_DDRAM_ADDRESS_SET_Msk (1 << INSTRUCTION_DDRAM_ADDRESS_SET_Pos)

#define DISPLAY_DELAY 3000
#define DELAY_SETUP_TIME 1
#define LINE_LENGTH 16

enum Line {
  FIRST,
  SECOND
};

void LCD_print(const char *message, enum Line line);
void LCD_init(bool dual_line, bool large_font, bool cursor_on, bool cursor_blink, bool increment_mode, bool display_shift_on);
void LCD_clear();

#endif
```

Core/Inc/led.h

```h
// led.h
#ifndef led_h
#define led_h

void LED_turn_off();
void LED_turn_on();

#endif
```

Core/Inc/main.h

```
/* USER CODE BEGIN Header */
/**
  ******************************************************************************
  * @file           : main.h
  * @brief          : Header for main.c file.
  *                   This file contains the common defines of the application.
  ******************************************************************************
  * @attention
  *
  * Copyright (c) 2026 STMicroelectronics.
  * All rights reserved.
  *
  * This software is licensed under terms that can be found in the LICENSE file
  * in the root directory of this software component.
  * If no LICENSE file comes with this software, it is provided AS-IS.
  *
  ******************************************************************************
  */
/* USER CODE END Header */

/* Define to prevent recursive inclusion -------------------------------------*/
#ifndef __MAIN_H
#define __MAIN_H

#ifdef __cplusplus
extern "C" {
#endif

/* Includes ------------------------------------------------------------------*/
#include "stm32l4xx_hal.h"
#include "stm32l4xx_nucleo.h"

/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */

/* USER CODE END Includes */

/* Exported types ------------------------------------------------------------*/
/* USER CODE BEGIN ET */

/* USER CODE END ET */

/* Exported constants --------------------------------------------------------*/
/* USER CODE BEGIN EC */

/* USER CODE END EC */

/* Exported macro ------------------------------------------------------------*/
/* USER CODE BEGIN EM */

/* USER CODE END EM */

/* Exported functions prototypes ---------------------------------------------*/
void Error_Handler(void);

/* USER CODE BEGIN EFP */

/* USER CODE END EFP */

/* Private defines -----------------------------------------------------------*/
#define USART_TX_Pin GPIO_PIN_2
#define USART_TX_GPIO_Port GPIOA
#define USART_RX_Pin GPIO_PIN_3
#define USART_RX_GPIO_Port GPIOA
#define TMS_Pin GPIO_PIN_13
#define TMS_GPIO_Port GPIOA
#define TCK_Pin GPIO_PIN_14
#define TCK_GPIO_Port GPIOA
#define SWO_Pin GPIO_PIN_3
#define SWO_GPIO_Port GPIOB

/* USER CODE BEGIN Private defines */

/* USER CODE END Private defines */

#ifdef __cplusplus
}
#endif

#endif /* __MAIN_H */
```

Core/Src/CMakeLists.txt

```
target_sources(${CMAKE_PROJECT_NAME} PRIVATE init.c keypad.c lcd.c led.c main.c)
```

Core/Src/init.c

```c
//init_c
#include "init.h"
#include "stdbool.h"
#include "stm32l476xx.h"
#include "stm32l4xx_hal.h"
#include <stdint.h>

// Function prototypes
void lcd_gpio_init();
void keypad_gpio_init();
void led_gpio_init();
void bus_init();

// Init every pin for this project
void gpio_init()
{
  HAL_Init();
  bus_init();
  lcd_gpio_init();
  keypad_gpio_init();
  led_gpio_init();
}

// Start the 2 hearts
void bus_init()
{
  RCC->AHB2ENR |= (RCC_AHB2ENR_GPIOCEN);
  RCC->AHB2ENR |= (RCC_AHB2ENR_GPIOAEN);
}

// LCD panel will use pines GPIOC 0-9
void lcd_gpio_init()
{
  //LCD Panel
  //Set PC0-9 as input mode
  GPIOC->MODER &= ~(GPIO_MODER_MODE0 | GPIO_MODER_MODE1 | GPIO_MODER_MODE2 | GPIO_MODER_MODE3);
  GPIOC->MODER &= ~(GPIO_MODER_MODE4 | GPIO_MODER_MODE5 | GPIO_MODER_MODE6);
  GPIOC->MODER &= ~(GPIO_MODER_MODE7 | GPIO_MODER_MODE8 | GPIO_MODER_MODE9);

  GPIOC->MODER |= (GPIO_MODER_MODE0_0 | GPIO_MODER_MODE1_0 | GPIO_MODER_MODE2_0 | GPIO_MODER_MODE3_0);
  GPIOC->MODER |= (GPIO_MODER_MODE4_0 | GPIO_MODER_MODE5_0 | GPIO_MODER_MODE6_0);
  GPIOC->MODER |= (GPIO_MODER_MODE7_0 | GPIO_MODER_MODE8_0 | GPIO_MODER_MODE9_0);

  return;
}

// Keypad will use pins GPIOA 5-8 and GPIOC 10-12
void keypad_gpio_init()
{
  // set rows as input
  // PC0-3
  GPIOA->MODER &= ~(GPIO_MODER_MODE5 | GPIO_MODER_MODE6 | GPIO_MODER_MODE7 | GPIO_MODER_MODE8);

  // set rows as pull-down
  GPIOA->PUPDR &= ~(GPIO_PUPDR_PUPD5 | GPIO_PUPDR_PUPD6 | GPIO_PUPDR_PUPD7 | GPIO_PUPDR_PUPD8);
  GPIOA->PUPDR |= (GPIO_PUPDR_PUPD5_1 | GPIO_PUPDR_PUPD6_1 | GPIO_PUPDR_PUPD7_1 | GPIO_PUPDR_PUPD8_1);

  // set cols as output
  // PC10-12
  GPIOC->MODER &= ~(GPIO_MODER_MODE10 | GPIO_MODER_MODE11 | GPIO_MODER_MODE12);
  GPIOC->MODER |= (GPIO_MODER_MODE10_0 | GPIO_MODER_MODE11_0 | GPIO_MODER_MODE12_0);
}

//LED will use pin GPIOB pin 0
void led_gpio_init()
{
  // set LED as output
  GPIOA->MODER &= ~(GPIO_MODER_MODE4);
  GPIOA->MODER |= (GPIO_MODER_MODE4_0);
}
```

Core/Src/keypad.c

```c
#include "keypad.h"
#include "stdbool.h"
#include "stm32l476xx.h"
#include <stdint.h>

/* On the Keypad, from left to right: Row1, Row2, Row3, Row4, Col.1, Col.2, Col.3
*/

#define ROW_COUNT 4
#define COL_COUNT 3
#define PA_ROW_START 5
#define PC_COLUMN_START 10
#define PA_LED_START 5

bool row_is_high(uint8_t i)
{
  return GPIOA->IDR & (1 << (PA_ROW_START + i));
}

void col_set(uint8_t j, bool high)
{
  GPIOC->ODR &= ~(1 << (PC_COLUMN_START + j));
  if (high)
    GPIOC->ODR |= (1 << (PC_COLUMN_START + j));
}

void cols_set_low()
{
  for (uint8_t j = 0; j < COL_COUNT; j++) {
    col_set(j, false);
  }
}

void col_isolate(uint8_t j)
{
  cols_set_low();
  col_set(j, true);
}

bool key_is_pressed(uint8_t row, uint8_t col)
{
  col_isolate(col);
  return row_is_high(row);
}

int get_key_value(uint8_t row, uint8_t col)
{
  // treat all rows but the last one as a regularly ordered numerical grid
  if (row < ROW_COUNT - 1)
    return (row * COL_COUNT) + col + 1;

  // treat the last row with special hardcoded values
  if (col == 0)
    return 10;

  if (col == 1)
    return 0;

  if (col == 2)
    return 15;

  return INVALID_VALUE;
}

int get_pressed_key()
{
  for (uint8_t i = 0; i < ROW_COUNT; i++)
    for (uint8_t j = 0; j < COL_COUNT; j++)
      if (key_is_pressed(i, j))
        return get_key_value(i, j);

  return INVALID_VALUE;
}
```

Core/Src/lcd.c

```c
#include "lcd.h"
#include "stdbool.h"
#include "stm32l476xx.h"
#include "stm32l4xx_hal.h"
#include <stddef.h>
#include <stdint.h>
#include <string.h> // For strlen();

/* PC1-8 = DB0-7 */
void DB_set(uint8_t val)
{
  GPIOC->ODR &= ~(0b11111111 << GPIO_ODR_OD1_Pos);
  GPIOC->ODR |= (val << GPIO_ODR_OD1_Pos);
}

/* PC9 = RS */
void RS_set(bool val)
{
  GPIOC->ODR &= ~GPIO_ODR_OD9_Msk;
  GPIOC->ODR |= (val << GPIO_ODR_OD9_Pos);
}

/* PC0 = E */
void E_set(bool val)
{
  GPIOC->ODR &= ~GPIO_ODR_OD0_Msk;
  GPIOC->ODR |= val << GPIO_ODR_OD0_Pos;
}

void bus_write(uint8_t val, bool set_RS)
{
  E_set(false);
  HAL_Delay(1);
  RS_set(set_RS);
  HAL_Delay(1);
  E_set(true);
  HAL_Delay(DELAY_SETUP_TIME);
  DB_set(val);
  HAL_Delay(DELAY_SETUP_TIME);
  E_set(false);
  HAL_Delay(DELAY_SETUP_TIME);
}

void instruction_send(uint8_t val)
{
  bus_write(val, false);
}

void data_send(uint8_t val)
{
  bus_write(val, true);
}

uint8_t instruction_mask_create(uint8_t offset, uint8_t argument_count, bool *arguments)
{
  uint8_t mask = 0;
  mask |= (1 << offset);

  for (uint8_t i = 0; i < argument_count; i++) {
    uint8_t argument_offset = offset - 1 - i;
    mask |= (arguments[i] << argument_offset);
  }

  return mask;
}

void LCD_clear()
{
  instruction_send(INSTRUCTION_DISPLAY_CLEAR_Msk);
}

void entry_mode_set(bool increment, bool display_shift_on)
{
  static const uint8_t argument_count = 2;
  bool arguments[argument_count];

  arguments[0] = increment;
  arguments[1] = display_shift_on;

  uint8_t mask = instruction_mask_create(INSTRUCTION_ENTRY_MODE_SET_Pos, argument_count, arguments);

  instruction_send(mask);
}

void display_on_off(bool display_on, bool cursor_on, bool cursor_blink)
{
  static const uint8_t argument_count = 3;
  bool arguments[argument_count];

  arguments[0] = display_on;
  arguments[1] = cursor_on;
  arguments[2] = cursor_blink;
```

```
  uint8_t mask = instruction_mask_create(INSTRUCTION_DISPLAY_ON_OFF_Pos, argument_count, arguments);

  instruction_send(mask);
}

void function_set(bool byte_mode, bool dual_line, bool font)
{
  static const uint8_t argument_count = 3;
  bool arguments[argument_count];

  arguments[0] = byte_mode;
  arguments[1] = dual_line;
  arguments[2] = font;

  uint8_t mask = instruction_mask_create(INSTRUCTION_FUNCTION_SET_Pos, argument_count, arguments);

  instruction_send(mask);
}

void ddram_address_set(uint8_t addr) {
  uint8_t mask = INSTRUCTION_DDRAM_ADDRESS_SET_Msk | addr;

  instruction_send(mask);
}

void line_set(bool second_line) {
  if (second_line)
    ddram_address_set(DDRAM_ADDRESS_SECOND_LINE);
  else
    ddram_address_set(DDRAM_ADDRESS_FIRST_LINE);
}

// TODO: need to have display ON/OFF configured
void LCD_write_char(uint8_t letter)
{
  data_send(letter);
}

void LCD_print(const char *message, enum Line line)
{
  size_t length = strlen(message);
  if (length > LINE_LENGTH) {
    message = "BAD LENGTH";
  }

  if (line == FIRST)
    line_set(false);
  else
    line_set(true);

  for (int i = 0; i < length; i++) {
    LCD_write_char(message[i]);
  }

  return;
}


void LCD_init(bool dual_line, bool large_font, bool cursor_on, bool cursor_blink, bool increment_mode, bool display_shift_on)
{
  function_set(true, dual_line, large_font);
  HAL_Delay(100);
  function_set(true, dual_line, large_font);
  HAL_Delay(100);
  function_set(true, dual_line, large_font);
  HAL_Delay(100);
  display_on_off(true, cursor_on, cursor_blink);
  HAL_Delay(100);
  LCD_clear();
  HAL_Delay(100);
  entry_mode_set(increment_mode, display_shift_on);
}

// TODO: check validity
```

Core/Src/led.c

```c
// led.c
#include "led.h"
#include "stdbool.h"
#include "stm32l476xx.h"
#include <stdint.h>

void LED_turn_off()
{
  GPIOA->ODR &= ~GPIO_ODR_OD4;
}

void LED_turn_on() {
    GPIOA->ODR |= GPIO_ODR_OD4;
}
```